

Go – Cours 3

API (Gin), GORM, et gestion de configuration

Programme du jour

FINIR TP GoAnalyser

Part 1

Cours

Part 2

TP - Analyse d'une API

Part 3

Q&A

Part 4

FIN DU TP



COURS



RAPPELS DE BASE



Qu'est-ce qu'une API ?

- **API = Application Programming Interface**
- Communication standardisée entre applications
- Exemples :
 - Appli mobile ↔ serveur
 - Site e-commerce ↔ système de paiement

REST, c'est quoi ?

- REST = **Representational State Transfer**
- Style d'architecture (pas un protocole)
- Inventé par Roy Fielding (2000)
- Objectif : services web simples, stateless, performants et évolutifs

Principes REST (1/2)

1. Client-Serveur : séparation des responsabilités
2. Stateless : aucune mémoire côté serveur
3. Cacheable : améliore les performances

Principes REST (2/2)

- 4. Layered System : communication via des intermédiaires
- 5. Interface Uniforme (4 contraintes) :
 - Ressources via URI
 - Représentation manipulable (JSON, XML)
 - Messages auto-descriptifs
 - Hypermedia as the Engine of Application State (HATEOAS) (liens dans les réponses)

Les verbes HTTP

Verbe	Action	Idempotent ?	Exemple
GET	Lire une ressource	✅ Oui	GET /products
POST	Créer une ressource	❌ Non	POST /products
PUT	Remplacer une ressource	✅ Oui	PUT /products/123
PATCH	Modifier partiellement	⚠️ Variable	PATCH /products/123
DELETE	Supprimer une ressource	✅ Oui	DELETE /products/123

Codes de Statut HTTP (2xx)

- **Série 2xx - Succès** : La requête a été reçue, comprise et acceptée.
 - **200 OK** : Requête traitée avec succès. C'est le code de succès le plus courant pour GET, PUT, PATCH, DELETE.
 - **201 Created** : La requête a été exécutée avec succès et une nouvelle ressource a été créée (typiquement après un POST). La réponse doit inclure l'URI de la nouvelle ressource dans l'en-tête **Location**.
 - **204 No Content** : La requête a été traitée avec succès, mais il n'y a aucune information à renvoyer dans le corps de la réponse (typiquement après un DELETE réussi).

Codes HTTP (4xx - Erreurs Client)

- **Série 4xx - Erreur Client** : La requête contient une erreur ou ne peut pas être traitée en raison d'un problème côté client.
 - **400 Bad Request** : La requête n'a pas pu être comprise par le serveur en raison d'une syntaxe invalide ou de données manquantes/incorrectes.
 - **401 Unauthorized** : La requête nécessite une authentification.
 - **403 Forbidden** : Le client n'a pas les permissions nécessaires pour accéder à la ressource.
 - **404 Not Found** : La ressource demandée n'existe pas sur le serveur.
 - **405 Method Not Allowed** : La méthode HTTP utilisée n'est pas supportée pour cette ressource.
 - **409 Conflict** : La requête ne peut pas être complétée en raison d'un conflit avec l'état actuel de la ressource (ex: tentative de créer une ressource qui existe déjà).
 - **429 Too Many Requests** : Le client a envoyé trop de requêtes dans un laps de temps donné (limitation de débit).

Codes HTTP (5xx - Erreurs Serveur)

- **Série 5xx - Erreur Serveur** : Le serveur a rencontré une erreur lors du traitement de la requête.
 - **500 Internal Server Error** : Erreur générique du serveur qui n'est pas couverte par un autre code d'erreur 5xx.
 - **502 Bad Gateway** : Le serveur, agissant comme une passerelle, a reçu une réponse invalide d'un serveur en amont.
 - **503 Service Unavailable** : Le serveur n'est pas prêt à gérer la requête (ex: surcharge, maintenance).

Règles de bonnes pratiques

- Utiliser le bon verbe HTTP
- Renvoyer le code de statut le plus précis
- Respecter les principes REST pour la scalabilité
- Documenter l'API (Swagger, Postman, etc.)

Pourquoi une API en Go ?

Encore des rappels !



Performance et Efficacité

- Langage compilé → rapide comme C/C++
- Moins gourmand en mémoire et CPU
- Idéal pour les charges web et API intensives

Concurrence Native

- goroutines = threads légers intégrés
- channels = communication entre routines
- Go gère des milliers de requêtes simultanées sans surcharge
- API hautement scalables avec peu de complexité

Robustesse et Typage Fort

- Typage statique : détection des erreurs à la compilation
- Gestion des erreurs explicite et obligatoire
- Encourage un code fiable et maintenable

Déploiement Ultra Simple

- Compilation rapide → gain de temps
- Génère un binaire autonome
 - Pas de dépendances externes
 - Un seul fichier à déployer sur le serveur

Créer une API avec Gin



Pourquoi Gin ?

- Gin = micro-framework Go pour APIs performantes et lisibles.
- Très rapide
- Routage avancé
- Middlewares intégrés
- Validation facile
- Syntaxe claire
- JSON natif

Installation de Gin

Pour l'installer dans votre projet, ouvrez votre terminal dans le répertoire de votre module Go et exécutez :

```
go get github.com/gin-gonic/gin
```

Serveur minimal

```
package main

import (
    "net/http" // Standard package pour les constantes HTTP
    "github.com/gin-gonic/gin" // Le framework Gin
)

func main() {
    // Initialise un routeur Gin avec les middlewares par défaut (Logger et Recovery)
    router := gin.Default()

    // Définit une route GET sur le chemin "/ping"
    // Quand une requête GET arrive sur /ping, le handler est exécuté
    router.GET("/ping", func(c *gin.Context) {
        // Renvoie une réponse JSON avec le statut HTTP 200 (OK)
        c.JSON(http.StatusOK, gin.H{
            "message": "pong", // gin.H est un raccourci pour map[string]interface{}
        })
    })

    // Démarre le serveur sur le port 8080
    // router.Run() écoute par défaut sur ":8080"
    // Vous pouvez spécifier un autre port comme router.Run(":9090")
    router.Run()
}
```

gin.Context - L'objet clé

- Le gin.Context (souvent abrégé en c dans les handlers) est l'objet le plus important dans Gin.
- Il encapsule toutes les informations sur la requête HTTP entrante et fournit des méthodes pour construire la réponse HTTP sortante.

gin.Context - L'objet clé

- Accessible via `func(c *gin.Context)`
- **Récupérer les données :**
 - `c.Query("name")` → query string
 - `c.Param("id")` → path param
 - `c.GetHeader("X-Token")` → header
 - `c.BindJSON(&data)` → JSON → struct Go
- **Réponses faciles :**
 - `c.JSON(200, data)`
 - `c.String(200, "Hello %s", name)`
 - `c.AbortWithStatusJSON(401, errObj)`

Example POST + Validation

```
type Product struct {  
    Name string `json:"name" binding:"required"`  
    Price float64 `json:"price" binding:"required,gt=0"`  
}  
  
router.POST("/products", func(c *gin.Context) {  
    var newProduct Product  
    if err := c.BindJSON(&newProduct); err != nil {  
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})  
        return  
    }  
    // newProduct contient maintenant les données du corps de la requête  
    c.JSON(http.StatusCreated, newProduct)  
})
```

Routage Avancé (Path Params)

```
// GET /users/42
router.GET("/users/:id", func(c *gin.Context) {
    userID := c.Param("id") // Récupère la valeur de ':id'
    c.JSON(http.StatusOK, gin.H{"message": "User ID: " + userID})
})

// GET /products/category/electronics/item/smartphone
router.GET("/products/category/:category/item/:item", func(c *gin.Context) {
    category := c.Param("category")
    item := c.Param("item")
    c.JSON(http.StatusOK, gin.H{
        "category": category,
        "item":      item,
    })
})
```

Groupes de Routes

```
api := router.Group("/api/v1")
{
  api.GET("/users", getUsers)
  api.POST("/users", createUser)

  products := api.Group("/products")
  {
    products.GET("/", listProducts)
    products.GET("/:id", getProduct)
  }
}
```

Middlewares – Rappels

- Un middleware = une fonction exécutée avant ou après un handler.
 - Sert à gérer des logiques transversales à toutes les routes :
 - Logging
 - Authentification / Autorisation
 - Validation
 - Gestion d'erreurs
 - CORS, headers, etc.

Middlewares - Example simple

```
func MonLoggerMiddleware() gin.HandlerFunc {  
    return func(c *gin.Context) {  
        start := time.Now()  
        c.Next()  
        fmt.Printf("%s %s | %d | %s\n",  
            c.Request.Method,  
            c.Request.URL.Path,  
            c.Writer.Status(),  
            time.Since(start))  
    }  
}
```

```
router.Use(MonLoggerMiddleware())
```

Middlewares – Où les placer ?

```
router := gin.Default() // Inclut déjà Logger + Recovery
router.Use(MonLoggerMiddleware()) // Appliqué globalement

// Middleware appliqué à un groupe de routes
admin := router.Group("/admin", AuthMiddleware())
```



Persistance de données avec un ORM (Gorm)

ORM – Rappels

- **ORM = Object Relational Mapper**
- Permet d'utiliser des structs Go pour manipuler les tables SQL
- Évite d'écrire du SQL manuellement à chaque fois
- Avantages :
 - Code plus propre
 - Sécurité accrue (SQL injection)
 - Mapping bidirectionnel struct $\leftrightarrow$ table

Pourquoi GORM ?

- **GORM = ORM le plus populaire en Go**
- Support de SQLite, PostgreSQL, MySQL, etc.
 - Auto-migrations
 - Relations (has-one, has-many...)
 - Hooks, transactions
 - API intuitive

Installation de Gorm

Pour utiliser GORM avec SQLite, vous aurez besoin de deux packages : GORM lui-même et le driver spécifique à SQLite.

```
go get gorm.io/gorm  
go get gorm.io/driver/sqlite
```

Connexion à SQLite avec GORM

```
// DB est l'instance de la base de données que nous utiliserons
var DB *gorm.DB

// ConnectDB initialise la connexion à la base de données
func ConnectDB() {
    var err error
    // Ouvre la connexion à la base de données SQLite.
    // Le chemin "products.db" crée un fichier dans le répertoire courant si la DB n'existe pas
    DB, err = gorm.Open(sqlite.Open("products.db"), &gorm.Config{})
    if err != nil {
        log.Fatalf("Échec de la connexion à la base de données : %v", err)
    }

    log.Println("Connexion à la base de données SQLite réussie !")
}
```

Définir un modèle GORM

```
type Product struct {  
    gorm.Model  
    Name      string `json:"name" gorm:"type:varchar(100);not null"`  
    Description string `json:"description" gorm:"type:text"`  
    Price      float64 `json:"price" gorm:"type:decimal(10,2);not null"`  
    Stock      int     `json:"stock" gorm:"default:0"`  
}
```

gorm.Model : Un champ embarqué (embedded field) qui fournit automatiquement des colonnes utiles comme ID, CreatedAt, UpdatedAt, et DeletedAt (pour la suppression "douce" ou "soft delete"). C'est une convention très pratique.

Créer les tables avec AutoMigrate

- Migration automatique : `DB.AutoMigrate(&Product{})`
 - Cette fonction va créer les tables manquantes, ajouter les colonnes manquantes, et créer des index si nécessaire, en se basant sur vos structs Go.
 - Elle **ne supprimera pas** de colonnes ou de tables si vous les supprimez de vos structs (pour éviter la perte de données).
 - C'est très pratique pour le développement initial. Pour la production, des outils de migration plus robustes (comme `golang-migrate`) sont souvent préférés.

Créer les tables avec AutoMigrate

```
// Effectue la migration automatique pour la struct Product
// Cela va créer la table 'products' si elle n'existe pas.
err = DB.AutoMigrate(&models.Product{})
if err != nil {
    log.Fatalf("Échec de la migration de la base de données : %v", err)
}
log.Println("Migration de la base de données réussie !")
}
```

Opérations CRUD - Create

```
// Créer un nouveau produit
func CreateProduct(product *models.Product) error {
    result := database.DB.Create(product)
    return result.Error // Retourne nil si succès, ou l'erreur
}
```

- Insère une nouvelle ligne
- GORM remplit automatiquement les champs ID, CreatedAt, etc.

Opérations CRUD - READ (ALL)

```
// Récupérer tous les produits
func GetAllProducts() ([]models.Product, error) {
    var products []models.Product
    result := database.DB.Find(&products) // Find() récupère toutes les lignes
    return products, result.Error
}
```

- Récupère tous les produits de la table

Opérations CRUD - READ (by id)

```
// Récupérer un produit par ID
func GetProductByID(id uint) (*models.Product, error) {
    var product models.Product
    // First() récupère la première ligne correspondant à la condition
    result := database.DB.First(&product, id)
    if result.Error != nil {
        return nil, result.Error
    }
    return &product, nil
}
```

- First() : récupère le premier résultat correspondant à la condition

Opérations CRUD - UPDATE (ALL)

```
// Mettre à jour un produit existant (tous les champs)
func UpdateProduct(product *models.Product) error {
    // Save() va mettre à jour tous les champs non nuls,
    // et insérer si l'ID n'existe pas (upsert)
    result := database.DB.Save(product)
    return result.Error
}
```

- Save() met à jour tous les champs du produit
- Fonctionne comme un upsert (insère si non trouvé)

Opérations CRUD - UPDATE (Champs spécifique)

```
// Mettre à jour seulement certains champs
func UpdateProductFields(id uint, updates map[string]interface{}) error {
    result := database.DB.Model(&models.Product{}).Where("id = ?", id).Updates
    return result.Error
}
```

```
UpdateProductFields(5, map[string]interface{}{
    "Price": 9.99,
    "Stock": 10,
})
```

Opérations CRUD - DELETE (soft)

- **Suppression "douce" (Soft Delete)** : GORM, avec `gorm.Model`, ne supprime pas réellement la ligne mais remplit le champ `DeletedAt`

```
// Supprimer (soft delete) un produit par ID
func DeleteProduct(id uint) error {
    result := database.DB.Delete(&models.Product{}, id)
    return result.Error
}
```

Opérations CRUD - DELETE (permanent)

- Suppression permanente (Unscoped Delete)

```
// Supprimer un produit de manière permanente
func HardDeleteProduct(id uint) error {
    result := database.DB.Unscoped().Delete(&models.Product{}, id)
    return result.Error
}
```

- Unscoped() désactive la logique de "soft delete"
- La ligne est supprimée définitivement

Gérer les erreurs avec GORM

```
// Dans un handler ou une fonction de service :
product, err := GetProductByID(productID)
if err != nil {
    if errors.Is(err, gorm.ErrRecordNotFound) {
        // Le produit n'existe pas, renvoyer 404
        c.JSON(http.StatusNotFound, gin.H{"error": "Produit non trouvé"})
        return
    }
    // Autre erreur de base de données, renvoyer 500
    c.JSON(http.StatusInternalServerError, gin.H{"error": "Erreur serveur : "})
    return
}
// ... Traiter le produit
```

Les DTO

- **DTO = Data Transfer Object**
- Objet simple qui a pour unique but de transporter des données entre différentes couches d'une application ou entre des systèmes
- Ne contient généralement aucune logique métier, seulement des champs pour stocker des données

Pourquoi les DTO ?

- **Séparation des modèles de base de données / entités métier des modèles de présentation / API**
- **Sécurité et Exposition des Données** : on peut omettre des champs confidentiels (mots de passe, informations internes) de vos DTOs de réponse d'API.
- **Agrégation de données** : Un DTO peut combiner des données provenant de plusieurs modèles de base de données ou services pour présenter une vue unifiée et simplifiée à l'utilisateur.
- **Validation des entrées** : parfaits pour y ajouter des tags de validation afin de s'assurer que les données reçues sont conformes aux attentes avant de les traiter.

Avantage - Séparation des responsabilités

- **Modèle GORM (models.Product)**
 - Structure pour la base de données
 - Champs GORM : `gorm:"..."`, `DeletedAt`, etc.
- **DTO (ProductDTO)**
 - Structure pour l'API
 - Champs JSON : `json:"..."`
- **Plus clair, plus propre, plus maintenable**

Avantage – Sécurité

```
type User struct {  
    ID          uint  
    Email       string  
    PasswordHash string  
    DeletedAt   gorm.DeletedAt  
}
```

- Si on renvoie directement ce modèle, le mot de passe hashé et DeletedAt sont visibles. Avec un DTO, vous choisissez ce que vous exposez :

```
type UserDTO struct {  
    ID      uint   `json:"id"`  
    Email   string `json:"email"`  
}
```

Avantage – Flexibilité et versions

- Base de données $\neq$ API
- Vous pouvez modifier `models.Product` sans casser l'API
- Vous pouvez créer :
 - `v1.ProductDTO`
 - `v2.ProductDTO`pour gérer l'évolution de votre API

Avantage – Validation des requêtes

- Avec Gin, les DTO d'entrée peuvent être validés automatiquement :

```
type RegisterDTO struct {  
    Email    string `json:"email" binding:"required,email"`  
    Password string `json:"password" binding:"required,min=8"`  
}
```

Avantage – Lisibilité

- Quand vous voyez ceci :

```
func Register(c *gin.Context) {  
    var dto RegisterDTO  
    if err := c.ShouldBindJSON(&dto); err != nil {  
        // ...  
    }  
}
```

Vous savez immédiatement :

- Que cette struct vient de l'utilisateur
- Qu'elle est utilisée dans le contexte de l'API

Exemple de DTO

```
type ProductInputDTO struct {  
    Name  string `json:"name" binding:"required"`  
    Price float64 `json:"price" binding:"required,min=0"`  
}  
  
type ProductOutputDTO struct {  
    ID      uint   `json:"id"`  
    Name    string `json:"name"`  
    Price   float64 `json:"price"`  
}
```

- Entrée (validation) :ProductInputDTO
- Sortie (API) : ProductOutputDTO

Gestion de configuration avec viper



Viper, c'est quoi ?

- **Viper est une bibliothèque de configuration complète pour Go**
- Lit YAML, JSON, TOML, .env...
- Lit aussi les variables d'environnement
- Définition de valeurs par défaut
- Prise en charge de la surcharge (env > fichier > défaut)
- Lecture typée des valeurs
- Reload automatique (optionnel)

Installation de Viper

- A installer dans votre projet

```
go get github.com/spf13/viper
```

Définir des valeurs par défaut

Elles seront utilisées si aucune autre source de configuration ne les surcharge.

```
package config

import (
    "fmt"
    "log"

    "github.com/spf13/viper"
)

// InitConfig initialise Viper et charge la configuration
func InitConfig() {
    // 1. Définir des valeurs par défaut
    viper.SetDefault("server.port", 8080)
    viper.SetDefault("database.name", "products.db")
    viper.SetDefault("database.dsn", "./data/products.db") // Data Source Name
    viper.SetDefault("app.environment", "development")

    fmt.Println("Valeurs par défaut définies.")
}
```

Lire à partir d'un fichier de configuration

```
func InitConfig() {
    viper.SetDefault("server.port", 8080)
    viper.SetDefault("database.name", "products.db")
    viper.SetDefault("database.dsn", "./data/products.db")
    viper.SetDefault("app.environment", "development")

    // 2. Spécifier le nom du fichier de configuration (sans extension)
    viper.SetConfigName("config") // Le nom du fichier est "config.yaml", "config.json"

    // 3. Ajouter des chemins où Viper doit chercher le fichier
    viper.AddConfigPath(".") // Chercher dans le répertoire courant
    viper.AddConfigPath("./config") // Chercher dans un sous-répertoire "config"
    viper.AddConfigPath("/etc/appname/") // Chemin absolu pour les déploiements Linux

    // 4. Lire le fichier de configuration
    if err := viper.ReadInConfig(); err != nil {
        if _, ok := err.(viper.ConfigFileNotFoundError); ok {
            // Fichier de configuration non trouvé, ce n'est pas une erreur fatale
            log.Println("Fichier de configuration 'config.yaml' non trouvé, utilisation de la configuration par défaut")
        } else {
            // Erreur de lecture du fichier de configuration
            log.Fatalf("Erreur lors de la lecture du fichier de configuration: %v", err)
        }
    } else {
        log.Println("Fichier de configuration chargé avec succès.")
    }

    fmt.Printf("Environnement: %s\n", viper.GetString("app.environment"))
    fmt.Printf("Port du serveur: %d\n", viper.GetInt("server.port"))
}
```

Example

```
server:  
  port: 9000  
  
database:  
  name: mydb.db  
  dsn: "/var/lib/data.db"  
  
app:  
  environment: production
```

```
viper.GetInt("server.port") // → 9000
```

Accéder aux valeurs typées

Viper propose des méthodes pour récupérer les valeurs avec le type Go approprié :

- `viper.GetString("key")`
- `viper.GetInt("key")`
- `viper.GetBool("key")`
- `viper.GetFloat64("key")`
- `viper.GetDuration("key")`
- `viper.GetTime("key")`
- `viper.GetStringSlice("key")` (pour un tableau de chaînes)
- `viper.Get("key")` (retourne `interface{}`, à caster manuellement)

Q&A

